DML

DATA MANIPULATION LANGUAGE

COMMAND: SELECT

Retrieval Queries in SQL : SELECT statement

SQL has one basic statement for retrieving information from a database

Retrieval Queries in SQL (cont.)

SELECT <attribute list>
FROM <table list>
WHERE <condition>

- <attribute list> is a list of attribute names whose values are to be retrieved by the query
- <table list> is a list of the relation names required to process the query
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

Populated Database--Fig.5.6

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Simple SQL Queries (cont.)

Query: Retrieve the name and address of all employees.

```
SELECT      FNAME,MINIT, LNAME, ADDRESS
    FROM    EMPLOYEE ;
```

FNAME	MINIT	LNAME	ADDRESS
John	B	Smith	731 Fondren, Houston, TX
Franklin	T	Wong	638 Voss, Houston, TX
Alicia	J	Zelaya	3321 Castle, Spring, TX
Jennifer	S	Wallace	291 Berry, Bellaire, TX
Ramesh	K	Narayan	975 Fire Oak, Humble, TX
Joyce	A	English	5631 Rice, Houston, TX
Ahmad	V	Jabbar	980 Dallas, Houston, TX
James	E	Borg	450 Stone, Houston, TX

Simple SQL Queries (cont.)

Query: Retrieve the information of all employees.

```
SELECT      *
    FROM    EMPLOYEE ;
```

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

Simple SQL Queries

Query : Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

```
Q0:     SELECT   BDATE, ADDRESS
        FROM             EMPLOYEE
        WHERE    FNAME='John' AND MINIT='B'
    AND          LNAME='Smith';
```

BDATE	ADDRESS	
1965-01-09	731 Fondren, Houston, TX	

Operator: IS/IS NOT

Query: show employees who are not working in any department.

```
Select *
From employee
Where  dno IS NULL ;
```

USE OF DISTINCT: To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used

```
SELECT  SALARY
      FROM   EMPLOYEE;
```

SALARY
30000
40000
25000
43000
38000
25000
25000
55000

```
SELECT      DISTINCT SALARY
FROM  EMPLOYEE;
```

SALARY
30000
40000
25000
43000
38000
55000

USE OF DISTINCT: To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used

```
SELECT DISTINCT agent_code,ord_amount
 FROM orders
  WHERE agent_code='A002';
```

AGENT_CODE	ORD_AMOUNT	CUST_CODE	ORD_NUM
A002	4000	C00022	200113
A002	2500	C00005	200106
A002	500	C00022	200123
A002	500	C00009	200120
A002	500	C00022	200126
A002	3500	C00009	200128
A002	1200	C00009	200133

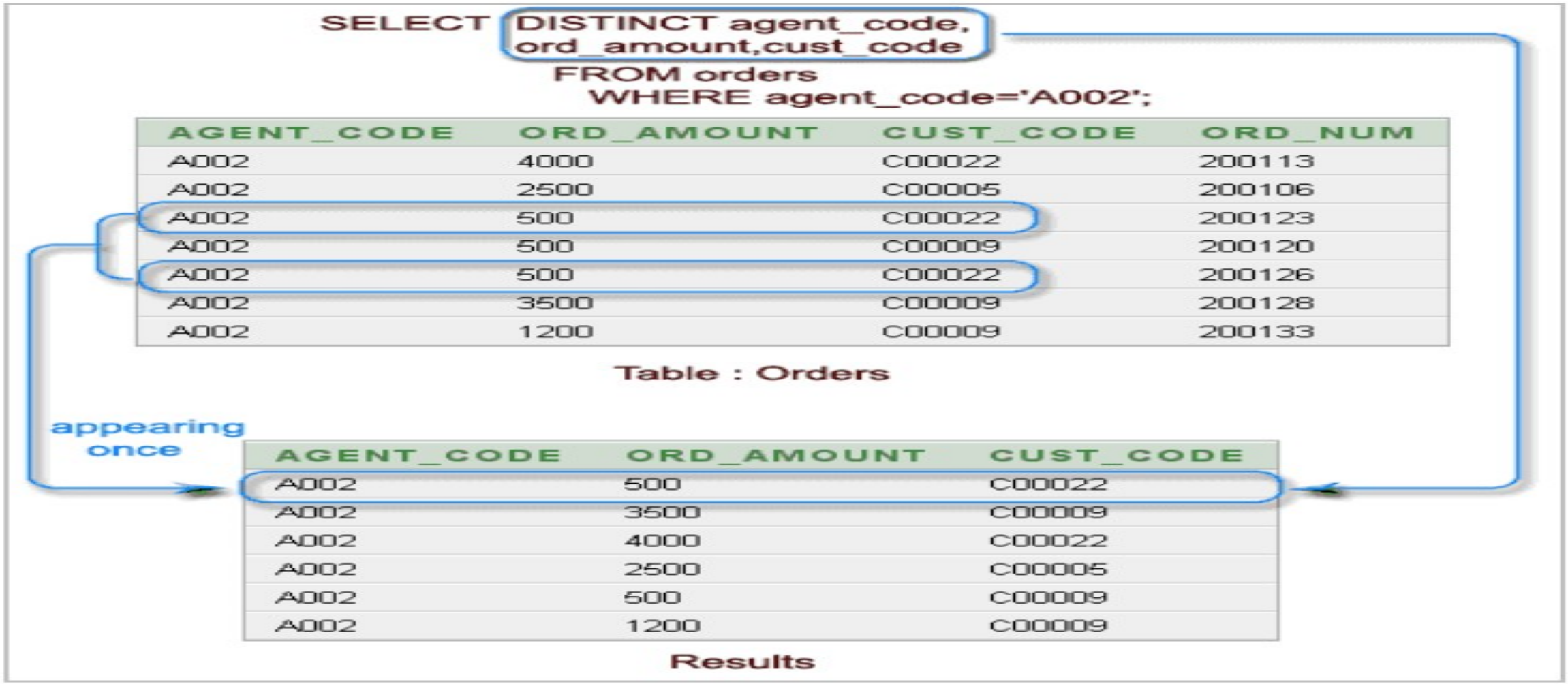
Table : Orders

appearing once

AGENT_CODE	ORD_AMOUNT
A002	3500
A002	4000
A002	1200
A002	500
A002	2500

Results

USE OF DISTINCT: To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used



AGENT_CODE	ORD_AMOUNT	CUST_CODE	ORD_NUM
A002	4000	C00022	200113
A002	2500	C00005	200106
A002	500	C00022	200123
A002	500	C00009	200120
A002	500	C00022	200126
A002	3500	C00009	200128
A002	1200	C00009	200133

AGENT_CODE	ORD_AMOUNT	CUST_CODE
A002	500	C00022
A002	3500	C00009
A002	4000	C00022
A002	2500	C00005
A002	500	C00009
A002	1200	C00009

ORDER BY

The **ORDER BY** clause is used to sort the tuples in a query result based on the values of some attribute(s)

The default order is in ascending order of values

We can specify the keyword **DESC** if we want a descending order; the keyword **ASC** can be used to explicitly specify ascending order, even though it is the default

ORDER BY (cont.)

Show all employee names in ascending order of their names.

```
Select Fname , Minit, Lname

From employee

Order by Fname;
```

ORDER BY (cont.)

Show all employee names in descending order of their names.

```
Select Fname , Minit, Lname
From employee
Order by Fname desc;
```

Comparison Operator BETWEEN

Query:

Retrieve all employees in department 5 whose salary is between 30,000 and 50,000

```
SELECT *
FROM EMPLOYEE
WHERE  (Salary BETWEEN 30000 AND 50000) AND DNO =5 ;
```

SAME AS

```
(Salary >= 30000)AND (Salary <= 50000)
```

ARITHMETIC OPERATIONS

The standard arithmetic operators '+', '-'. '*', and '/' can be applied to numeric values in an SQL query result

Query Show the effect of giving all employees who work in department no 5 a 10% raise.

```
Q:      SELECT   FNAME, LNAME, 1.1*SALARY
        FROM     EMPLOYEE
        WHERE    dno =5;
```

ARITHMETIC OPERATIONS Example

1. sum of 'opening_amt' and 'receive_amt' is greater than 15000,

the following SQL statement can be used :

```
1  SELECT cust_name, opening_amt,
2  receive_amt, (opening_amt + receive_amt)
3  FROM customer
4  WHERE (opening_amt + receive_amt)>15000;
```

Output:

```
CUST_NAME                                OPENING_AMT RECEIVE_AMT (OPENING_AMT+RECEIVE_AMT)
---------------------------------------- ----------- ----------- -------------------------
Sasikant                                        7000       11000                     18000
Ramanathan                                      7000       11000                     18000
Avinash                                         7000       11000                     18000
Shilton                                        10000        7000                     17000
Rangarappa                                      8000       11000                     19000
Venkatpati                                      8000       11000                     19000
Sundariya                                       7000       11000                     18000
```

Practice operators: <,>,<=,>=, <>

1. Show fname of all employees.
2. Show fname and ssn of all employees.
3. Show fname, ssn and salary of all employees who earns 40,000.
4. Show fname, ssn and salary of all employees who earns more than 40,000 and are working in dnumber 5.
5. Show fname of emloyees who earns 30,000 or they are female.
6. Show information of employees who have no supervisor.
7. Show fname of employees whose address is saved in our database.
8. Show uniques fname of all employees.
9. Show employees who are working in department # 5 or earn between 20,000 and 40,000
10. Show information of all employees in descending order by their salaries.

SUBSTRING COMPARISON

The **LIKE** comparison operator is used to compare partial strings

- reserved characters used: '%’ , ‘_’
- % replaces an arbitrary number of characters i.e. 0 to n
- '_' replaces a single arbitrary character i.e. only 1

SUBSTRING COMPARISON

The **LIKE** comparison operator is used to compare partial strings

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

SUBSTRING COMPARISON (cont.)

Retrieve all employees whose address is in Houston, Texas.

Here, the value of the ADDRESS attribute must contain the substring 'Houston,TX'.

```
SELECT      FNAME, LNAME
FROM        EMPLOYEE
WHERE       ADDRESS LIKE  '%Houston,TX%'
```

Like Example

Example: MySQL LIKE operator

The following MySQL statement scans the whole *author* table to find any author name which has a first name starting with character 'W' followed by any characters.

Code:

```
1  SELECT aut_name, country
2  FROM author
3  WHERE aut_name LIKE 'W%';
```

Like Example

The following MySQL statement scans the whole *author* table to find any author which have a string 'an' in his name. Name of the author is stored in aut_name column.

Code:

```
1  SELECT aut_name, country
2  FROM author
3  WHERE aut_name LIKE '%an%';
```

Like Example

The following MySQL statement scans the whole *author* table to find any author which have a string 'an' in his name. Name of the author is stored in aut_name column.

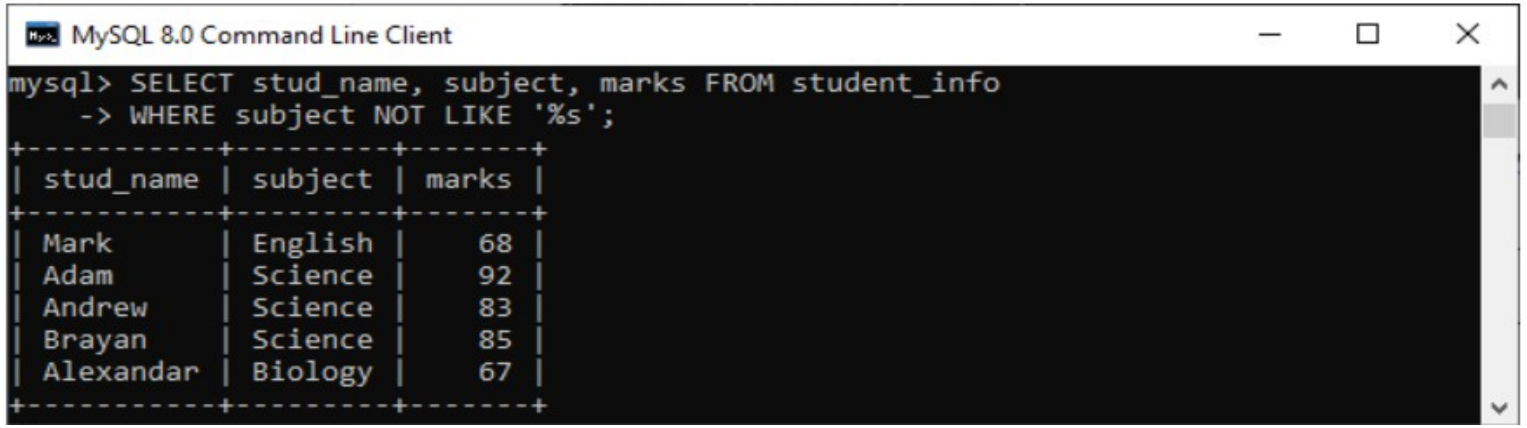
Code:

```
1  SELECT aut_name, country
2  FROM author
3  WHERE aut_name LIKE '%an%';
```

Not Like Example

```
mysql> SELECT stud_name, subject, marks FROM student_info
WHERE subject NOT LIKE '%s';
```

After executing this statement, we will get the desired result:



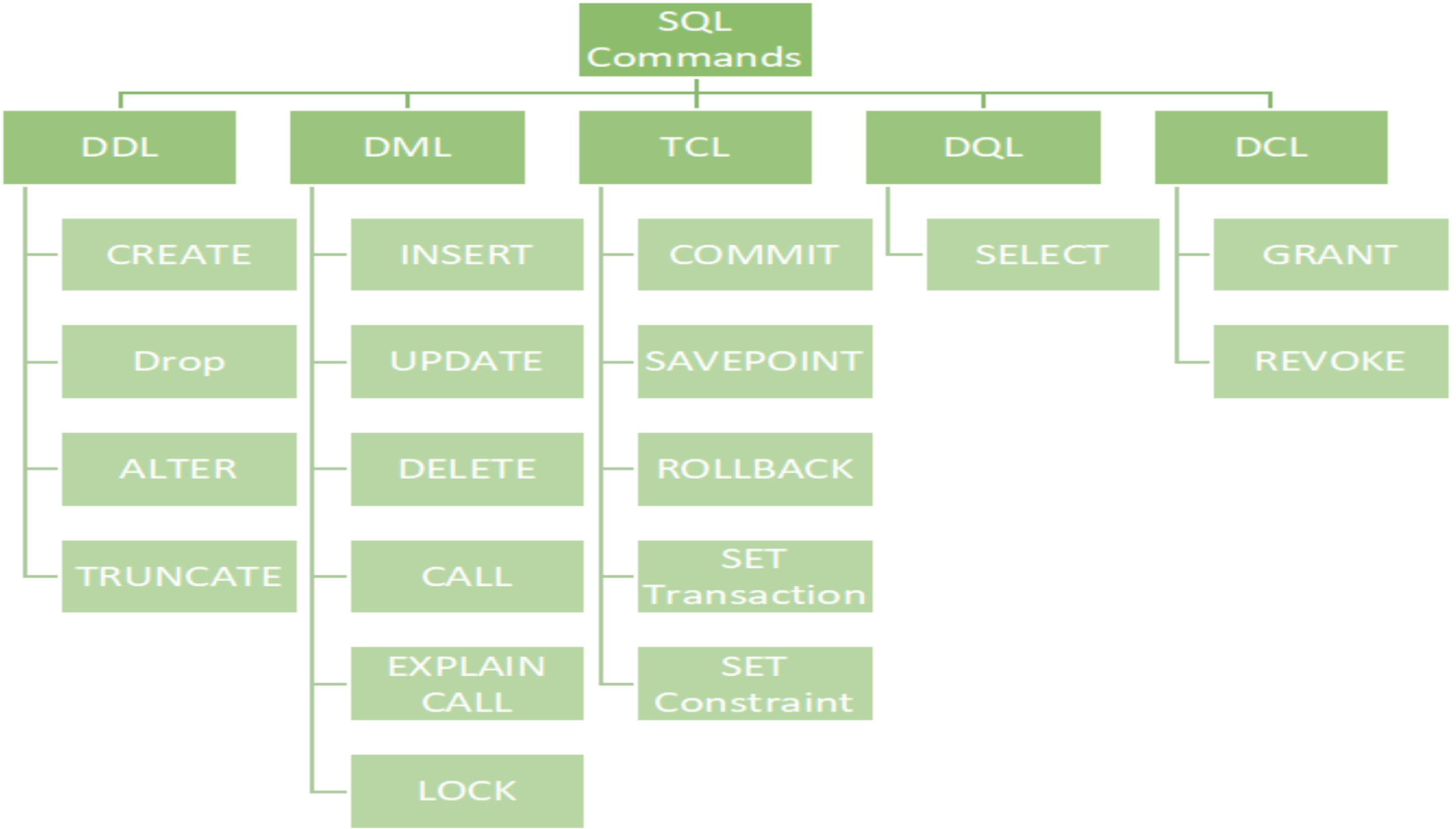
Practice Queries : LIKE / NOT LIKE

1. Show detail of employee whose first name starts with K and born in 1980.
2. Show detail of employee whose first name starts with J and last name does not end with d.
3. Show projects which has r in their name and it is on second position.
4. Show projects which does not has H in their name.
5. Show projects which does not has H on second position of their name.
6. Show name of dependent from dependent table, where name starts with D and has at least 3 characters.
7. Show detail of employee whose first name starts with A and ends with A.
8. Shaw name of employees who live in “Texas”.

DML

DATA MANIPULATION LANGUAGE

COMMANDS: INSERT, DELETE, UPDATE & SELECT


SQL Commands
DDL
DML
TCL
DQL
DCL
CREATE
Drop
ALTER
TRUNCATE
INSERT
UPDATE
DELETE
CALL
EXPLAIN CALL
LOCK
COMMIT
SAVEPOINT
ROLLBACK
SET Transaction
SET Constraint
SELECT
GRANT
REVOKE

Specifying Updates in SQL

There are three SQL commands to modify the database; INSERT, DELETE, and UPDATE

INSERT

In its simplest form, it is used to add one or more tuples/rows to a relation/table

Attribute values should be listed in the same order as the attributes were specified in the CREATE TABLE command

INSERT

The basic syntax for creating a table can be given with:

1. INSERT INTO **table_name** (**column1, column2, column3,)**

VALUES (value1, value2, value3, ...);

(Above query will enter data in specific column)

2.**INSERT** INTO **table_name** VALUES (**value1, value2, value3, ...);** (Without Specifying Columns)

(If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table.)

INSERT (cont.)

Example:

```
INSERT INTO  EMPLOYEE
VALUES ('Richard','K','Marini', '653298653', '30-DEC-52',
'98 Oak Forest,Katy,TX', 'M', 37000,'987654321', 4 );
```

Attributes with NULL values can be left out

INSERT (cont.)

Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
    VALUES ('Richard', 'Marini', '653298653'), ('Ali', 'Ahmed' , '123456889');
```

DELETE

Removes tuples from a relation

Includes a WHERE-clause to select the tuples/rows to be deleted

Where conditions can be combined with **Logical** and **Comparison** Operators

DELETE (cont.)

Examples:

```
DELETE FROM  EMPLOYEE
WHERELNAME='Brown’;

DELETE FROM  EMPLOYEE
WHERESSN='123456789’;

DELETE FROM  EMPLOYEE
WHEREDNO  =5;
```

The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

Delete (cont.)

A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table

- **DELETE FROM EMPLOYEE;**

Delete (cont.)

Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)

Delete from employee

Where ssn = 123456789;

UPDATE

Used to modify attribute values of one or more selected tuples

A WHERE-clause selects the tuples to be modified

An SET-clause specifies the attributes to be modified and their new values

UPDATE (cont.)

Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

```
U5:   UPDATE     PROJECT
      SET        PLOCATION = 'Bellaire', DNUM = 5
      WHERE      PNUMBER=10;
```

UPDATE (cont.)

Example: Give all employees in the 'Research' department a 10% raise in salary.

U6: **UPDATE EMPLOYEE**
SET SALARY = SALARY *1.1
WHERE dno =5;

In this request, the modified SALARY value depends on the original SALARY value in each tuple

Retrieval Queries in SQL : SELECT statement

SQL has one basic statement for retrieving information from a database

Retrieval Queries in SQL (cont.)

SELECT <attribute list>
FROM <table list>
WHERE <condition>

- <attribute list> is a list of attribute names whose values are to be retrieved by the query
- <table list> is a list of the relation names required to process the query
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

Populated Database--Fig.5.6

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Simple SQL Queries (cont.)

Query: Retrieve the name and address of all employees.

```
SELECT    FNAME,MINIT, LNAME, ADDRESS
    FROM  EMPLOYEE ;
```

FNAME	MINIT	LNAME	ADDRESS
John	B	Smith	731 Fondren, Houston, TX
Franklin	T	Wong	638 Voss, Houston, TX
Alicia	J	Zelaya	3321 Castle, Spring, TX
Jennifer	S	Wallace	291 Berry, Bellaire, TX
Ramesh	K	Narayan	975 Fire Oak, Humble, TX
Joyce	A	English	5631 Rice, Houston, TX
Ahmad	V	Jabbar	980 Dallas, Houston, TX
James	E	Borg	450 Stone, Houston, TX

Simple SQL Queries (cont.)

Query: Retrieve the information of all employees.

```
SELECT      *
    FROM    EMPLOYEE ;
```

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

Simple SQL Queries

Query : Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

```
Q0:     SELECT   BDATE, ADDRESS
        FROM             EMPLOYEE
        WHERE    FNAME='John' AND MINIT='B'
    AND          LNAME='Smith';
```

BDATE	ADDRESS	
1965-01-09	731 Fondren, Houston, TX	

Operator: IS/IS NOT

Query: show employees who are not working in any department.

```
Select *
From employee
Where  dno IS NULL ;
```

USE OF DISTINCT: To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used

SELECT SALARY
FROM EMPLOYEE;

SALARY
30000
40000
25000
43000
38000
25000
25000
55000

SELECT DISTINCT SALARY
FROM EMPLOYEE;

SALARY
30000
40000
25000
43000
38000
55000

ORDER BY

The **ORDER BY** clause is used to sort the tuples in a query result based on the values of some attribute(s)

The default order is in ascending order of values

We can specify the keyword **DESC** if we want a descending order; the keyword **ASC** can be used to explicitly specify ascending order, even though it is the default

ORDER BY (cont.)

Show all employee names in ascending order of their names.

```
Select Fname , Minit, Lname

From employee

Order by Fname;
```

ORDER BY (cont.)

Show all employee names in descending order of their names.

```
Select Fname , Minit, Lname
From employee
Order by Fname desc;
```

Comparison Operator BETWEEN

Query:

Retrieve all employees in department 5 whose salary is between 30,000 and 50,000

```
SELECT *
FROM EMPLOYEE
WHERE  (Salary BETWEEN 30000 AND 50000) AND DNO =5 ;
```

SAME AS

```
(Salary >= 30000)AND (Salary <= 50000)
```

ARITHMETIC OPERATIONS

The standard arithmetic operators '+', '-'. '*', and '/' can be applied to numeric values in an SQL query result

Query Show the effect of giving all employees who work in department no 5 a 10% raise.

```
Q:      SELECT   FNAME, LNAME, 1.1*SALARY
        FROM     EMPLOYEE
        WHERE    dno =5;
```

Practice operators: <,>,<=,>=, <>

1. Show fname of all employees.
2. Show fname and ssn of all employees.
3. Show fname, ssn and salary of all employees who earns 40,000.
4. Show fname, ssn and salary of all employees who earns more than 40,000 and are working in dnumber 5.
5. Show fname of emloyees who earns 30,000 or they are female.
6. Show information of employees who have no supervisor.
7. Show fname of employees whose address is saved in our database.
8. Show uniques fname of all employees.
9. Show employees who are working in department # 5 or earn between 20,000 and 40,000
10. Show information of all employees in descending order by their salaries.

SUBSTRING COMPARISON

The **LIKE** comparison operator is used to compare partial strings

- reserved characters used: '%' , '_'
- % replaces an arbitrary number of characters i.e. 0 to n
- '_' replaces a single arbitrary character i.e. only 1

SUBSTRING COMPARISON (cont.)

Retrieve all employees whose address is in Houston, Texas.

Here, the value of the ADDRESS attribute must contain the substring 'Houston,TX'.

```
SELECT      FNAME, LNAME
FROM        EMPLOYEE
WHERE       ADDRESS LIKE  '%Houston,TX%'
```

SUBSTRING COMPARISON (cont.)

Retrieve all employees who were born during the 1950s.

Here, '5' must be the 3rd character of the string (according to our format for date), so the BDATE value is '__5_____', with each underscore as a place holder for a single arbitrary character.

Q26: **SELECT FNAME, LNAME**
FROM EMPLOYEE
WHERE BDATE LIKE '195 _ _ _ _ _ _ _'

Practice Queries : LIKE / NOT LIKE

1. Show detail of employee whose first name starts with K and born in 1980.
2. Show detail of employee whose first name starts with J and last name does not end with d.
3. Show detail of department which has at least 3 alphabets in its name.
4. Show projects which has r in their name and it is on second position.
5. Show projects which does not has H in their name.
6. Show projects which does not has H on second position of their name.
7. Show name of dependent from dependent table, where dependent name starts with D and has at least 3 characters.
8. Show detail of employee whose first name starts with A and ends with A.
9. Show name of employee who have 5 digit salary.
10. Shaw name of employees who live in “Texas”.

Show employee names with their dept names? 24 records - 8 records = 16 faulty records

Select fname, Dname (3)

From employee, department (1)

Where employee.dno = department.dnumber; (2) // we are joining tables pk = fk

2 tables = 1 join condition

3 tables= 2 join conditions

Salary >1000 (query condition)

OUTPUT

John Research

Franklin research

Show employee names with their dept names? 24 records - 8 records = 16 faulty records

Select fname, Dname (3)

From employee, department (1) cross product / cartesian product ??

Output

John Research

John Admin

John HQ

Franklin research

Franklin Admin

Franklin HQ….

James research

James admin

James hq

Query from multiple tables.

Example: Show employee sssn and the department number they are working on.

```
SELECT SSN, DNAME
FROM  EMPLOYEE, DEPARTMENT
```

- It is extremely important not to overlook specifying any selection and join conditions in the WHERE-clause; otherwise, incorrect and very large relations may result

Simple SQL Queries (Using two tables)

Query: Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT   fname, address
FROM     employee, department
WHERE    dname='research' and dno = dnumber;
```

Output

john

Franklin

Ramesh

Joyce

Simple SQL Queries (cont.)

Query: Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT   FNAME, LNAME, ADDRESS
FROM     EMPLOYEE, DEPARTMENT
WHERE    DNAME='Research' AND DNUMBER=DNO
```

Simple SQL Queries (cont.)

JOIN CONDITION PK=FK

QUERY CONDITION attribute = value

Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
 3  SELECT p.pnumber, p.dnum, e.lname, e.address, e.bdate
1   FROM  project p, department d , employee e
 2  WHERE p.dnum=d.dnumber and d.mgrssn = e.ssn and p.plocation =
Stafford;
```

```
10   4    987654321
30    4    987654321
```

Simple SQL Queries (cont.)

JOIN CONDITION PK=FK

QUERY CONDITION attribute = value

Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
3  SELECT p.pnumber, p.dnum, e.lname, e.address, e.bdate
1  FROM  project p, department d , employee e
2  WHERE p.dnum=d.dnumber and d.mgrssn = e.ssn and p.plocation =
Stafford;
```

```
3  SELECT p.pnumber, p.dnum, e.lname, e.address, e.bdate
1  FROM (project p join department d on dnum=dnumber)  join employee e
on mgrssn=ssn
2  WHERE plocation = 'Stafford'  ;
```

Simple SQL Queries (cont.)

Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
Q2:     SELECT  PNUMBER, DNUM, LNAME, BDATE, ADDRESS
        FROM            PROJECT, DEPARTMENT, EMPLOYEE
        WHERE   DNUM=DNUMBER AND MGRSSN=SSN             AND
                PLOCATION='Stafford'
```

Aliases

In SQL, we can use the same name for two (or more) attributes as long as the attributes are in *different relations*.

A query that refers to two or more attributes with the same name must *qualify* the attribute name with the relation name by *prefixing* the relation name to the attribute name

Practice

1. Show department name with its manager's name
2. Show project name with its department name.
3. Show employee name with its dependent name.
4. Show employee name with the name of project he is working on.
5. Show employee name with its supervisor name.
6. Display name of manager and the date he starting managing the department.
7. Show department name with its department location.
8. Name the employee who works on a project that is located in 'Stafford'.
9. Name the employees who work on the project that is controlled by dept 5 or he manages dept 5.
10. Name all employees who have a dependents with the same first name as theirs.

SQL (Alias)

You can rename a table or a column temporarily by giving another name known as **Alias**.

Syntax

The basic syntax of a table alias is as follows.

```
SELECT column1, column2....
FROM table_name AS alias_name
WHERE [condition];
```

The basic syntax of a column alias is as follows.

```
SELECT column_name AS alias_name
FROM table_name
WHERE [condition];
```

SQL (Alias Example)

Example

Consider the following two tables.

Table 1 – CUSTOMERS Table is as follows.

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

SQL (Alias Example)

Following is the usage of a **column alias**.

```
SQL> SELECT  ID AS CUSTOMER_ID, NAME AS CUSTOMER_NAME
   FROM CUSTOMERS
   WHERE SALARY IS NOT NULL;
```

This would produce the following result.

```
+-------------+---------------+
| CUSTOMER_ID | CUSTOMER_NAME |
+-------------+---------------+
|           1 | Ramesh        |
|           2 | Khilan        |
|           3 | kaushik       |
|           4 | Chaitali      |
|           5 | Hardik        |
|           6 | Komal         |
|           7 | Muffy         |
+-------------+---------------+
```

SQL (Intersection & Union Pre-Conditions)

Intersection:

The INTERSECT clause in SQL is used to combine two SELECT statements but the dataset returned by the INTERSECT statement will be the intersection of the data-sets of the two SELECT statements. In simple words, the INTERSECT statement will **return only those rows** which will be **common** to both of the SELECT statements.

To use this **UNION clause, each SELECT statement must have**

1. The number and order of columns in both queries has to be the same
2. The data types of corresponding columns from both the select queries must be compatible with each other

SQL (Intersection & Union Pre-Conditions)

Intersection:

The INTERSECT clause in SQL is used to combine two SELECT statements but the dataset returned by the INTERSECT statement will be the intersection of the data-sets of the two SELECT statements. In simple words, the INTERSECT statement will **return only those rows** which will be **common** to both of the SELECT statements.

To use this **UNION clause, each SELECT statement must have**

1. The number and order of columns in both queries has to be the same
2. The data types of corresponding columns from both the select queries must be compatible with each other

ID	Name	Country	City
1	Aakash	INDIA	Mumbai
2	George	USA	New York
3	David	INDIA	Bangalore
4	Leo	SPAIN	Madrid
5	Rahul	INDIA	Delhi
6	Brian	USA	Chicago
7	Justin	SPAIN	Barcelona

Customers

Branch_Code	Country	City
101	INDIA	Mumbai
201	INDIA	Bangalore
301	USA	Chicago
401	USA	New York
501	SPAIN	Madrid

Branches

```
SELECT Country, City
FROM Customers
INTERSECT
SELECT Country, City
FROM Branches
ORDER BY City;
```

Output:

Country	City
INDIA	Bangalore
USA	Chicago
SPAIN	Madrid
INDIA	Mumbai
USA	New York

SQL (Union vs Union All)

UNION and ***UNION ALL*** are SQL operators used to ***concatenate 2*** or more result sets.

The main difference between UNION and UNION ALL is that:

- **UNION:** only keeps *unique* records
- **UNION ALL:** keeps all records, including *duplicates*
- The **UNION** clause/operator is used to combine the results of two or more SELECT statements ***without returning any duplicate rows.***
- The ***UNION ALL*** operator is used to combine the results of two SELECT statements ***including duplicate rows.***

SQL (Union)

Employee ID	Name	Age
1001	Ajay	23
1002	Babloo	34
1003	Chhavi	35
1004	Dheeraj	35
1005	Evina	30

Manager ID	Name	Age
1001	Garima	21
1002	Fredy	23
1003	Hans	23
1004	Ivanka	34
1005	Jai	43

```
SELECT Age
FROM Employee
UNION
SELECT Age
FROM Manager;
```

Age
21
23
30
34
35
43

SQL (Union All)

If you had the *suppliers* table populated with the following records:

supplier_id	supplier_name
1000	Microsoft
2000	Oracle
3000	Apple
4000	Samsung

And the *orders* table populated with the following records:

order_id	order_date	supplier_id
1	2015-08-01	2000
2	2015-08-01	6000
3	2015-08-02	7000
4	2015-08-03	8000

```
SELECT supplier_id
FROM suppliers
UNION ALL
SELECT supplier_id
FROM orders
ORDER BY supplier_id;
```

You would get the following results:

supplier_id
1000
2000
2000
3000
4000
6000
7000
8000

SQL (Union All with Diff column Names)

If you had the *suppliers* table populated with the following records:

supplier_id	supplier_name
1000	Microsoft
2000	Oracle
3000	Apple
4000	Samsung

And the *orders* table populated with the following records:

order_id	order_date	supplier_id
1	2015-08-01	2000
2	2015-08-01	6000
3	2015-08-02	7000
4	2015-08-03	8000

And you executed the following UNION ALL statement:

```
SELECT supplier_id, supplier_name
FROM suppliers
WHERE supplier_id > 2000
UNION ALL
SELECT company_id, company_name
FROM companies
WHERE company_id > 1000
```

You would get the following results:

supplier_id	supplier_name
3000	Apple
3000	Apple
4000	Samsung
7000	Sony
8000	IBM

SQL (IN keyword)

The SQL IN Operator

The `IN` operator allows you to specify multiple values in a `WHERE` clause.

The `IN` operator is a shorthand for multiple `OR` conditions.

IN Syntax

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

In this example, we have a table called *suppliers* with the following data:

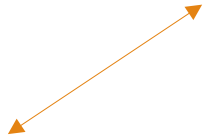
supplier_id	supplier_name	city	state
100	Microsoft	Redmond	Washington
200	Google	Mountain View	California
300	Oracle	Redwood City	California
400	Kimberly-Clark	Irving	Texas
500	Tyson Foods	Springdale	Arkansas
600	SC Johnson	Racine	Wisconsin
700	Dole Food Company	Westlake Village	California
800	Flowers Foods	Thomasville	Georgia
900	Electronic Arts	Redwood City	California

Enter the following SQL statement:

```
SELECT *
FROM suppliers
WHERE supplier_name IN ('Microsoft', 'Oracle', 'Flowers Foods');
```

There will be 3 records selected. These are the results that you should see:

supplier_id	supplier_name	city	state
100	Microsoft	Redmond	Washington
300	Oracle	Redwood City	California
800	Flowers Foods	Thomasville	Georgia



```
SELECT *
FROM suppliers
WHERE supplier_name = 'Microsoft'
OR supplier_name = 'Oracle'
OR supplier_name = 'Flowers Foods';
```

Not In

The **NOT IN** operator is used to replace a group of arguments using the **<> (or !=)** operator that are combined with an AND. It can make code easier to read and understand for SELECT, UPDATE or DELETE SQL commands. Generally, it will not change performance characteristics

```
SELECT *
FROM Sales
WHERE LastEditedBy NOT IN (11,17,13);
```

Query 1

```
SELECT *
FROM Sales
WHERE LastEditedBy <> 11
  AND LastEditedBy <> 17
  AND LastEditedBy <> 13;
```

Query 2

Both Query 1 and 2 are equal

Multi-table SQL Queries (Using two or more tables)

```
SELECT    fname, address
FROM      employee, department
WHERE     dname='research' and dno = dnumber;
```

More than one table can be specified in the select query , Like in this example there are two tables employee and department

Cross Product /Cartesian Product

It combines R1 and R2 without any condition. It is denoted by X.

Degree of R1 XR2 = degree of R1 + degree of R2

{degree = total no of columns}

Example

Consider R1 table –

RegNo	Branch	Section
1	CSE	A
2	ECE	B
3	CIVIL	A
4	IT	B

Table R2

Name	RegNo
Bhanu	2
Priya	4

Cross Product /Cartesian Product

```
SELECT *
FROM R1
CROSS JOIN R2;
```

R1 X R2

RegNo	Branch	Section	Name	RegNo
1	CSE	A	Bhanu	2
1	CSE	A	Priya	4
2	ECE	B	Bhanu	2
2	ECE	B	Priya	4
3	CIVIL	A	Bhanu	2
3	CIVIL	A	Priya	4
4	IT	B	Bhanu	2
4	IT	B	Priya	4

Cross Product /Cartesian Product

Cartesian Product(X) in DBMS

Cartesian Product in DBMS is an operation used to merge columns from two relations. Generally, a cartesian product is never a meaningful operation when it performs alone. However, it becomes meaningful when it is followed by other operations. It is also called Cross Product or Cross Join.

Example – Cartesian product

$\sigma_{\text{column 2}} = \text{'1'} (A \times B)$

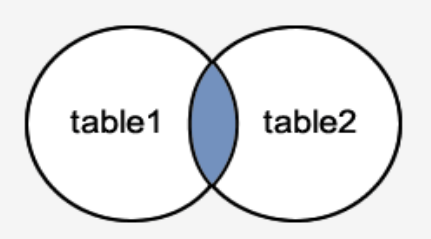
Output – The above example shows all rows from relation A and B whose column 2 has value 1

σ column 2 = '1' (A X B)	
column 1	**column 2**
1	1
1	1

INNER JOIN

The MySQL Inner Join is used to returns only those results from the tables that **match** the specified condition and hides other rows and columns. MySQL assumes it as a default Join, so it is optional to use the Inner Join keyword with the query.

We can understand it with the following visual representation where Inner Joins returns only the matching results from table1 and table2:



MySQL Inner Join Syntax:

The Inner Join keyword is used with the SELECT statement and must be written after the FROM clause. The following syntax explains it more clearly:

```
SELECT columns
FROM table1
INNER JOIN table2 ON condition1
INNER JOIN table3 ON condition2
...;
```

In this syntax, we first have to select the column list, then specify the table name that will be joined to the main table, appears in the Inner Join (table1, table2), and finally, provide the condition after the ON keyword. The Join condition returns the matching rows between the tables specifies in the Inner clause.

INNER JOIN

MySQL Inner Join Example

Let us first create two tables "students" and "technologies" that contains the following data:

Table: student

student_id	stud_fname	stud_lname	city
1	Devine	Putin	France
2	Michael	Clark	Australiya
3	Ethon	Miller	England
4	Mark	Strauss	America

Table: technologies

student_id	tech_id	inst_name	technology
1	1	Java Training Inst	Java
2	2	Chroma Campus	Angular
3	3	CETPA Infotech	Big Data
4	4	Aptron	IOS

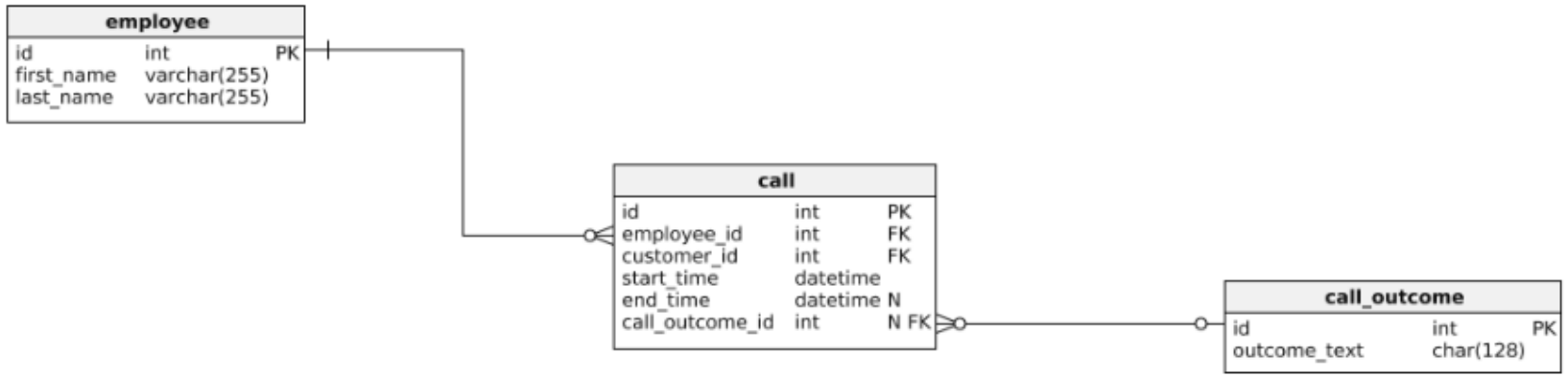
To select records from both tables, execute the following query:

```
SELECT students.stud_fname, students.stud_lname, students.city, technologies.technology
FROM students
INNER JOIN technologies
ON students.student_id = technologies.tech_id;
```

After successful execution of the query, it will give the following output:

stud_fname	stud_lname	city	technology
Devine	Putin	France	Java
Michael	Clark	Australiya	Angular
Ethon	Miller	England	Big Data
Mark	Strauss	America	IOS

INNER JOIN



The query that does the job is given below:

```
SELECT employee.first_name, employee.last_name, call.start_time, call.end_time, call_outcome.ou
tcome_text
FROM employee
INNER JOIN call ON call.employee_id = employee.id
INNER JOIN call_outcome ON call.call_outcome_id = call_outcome.id
ORDER BY call.start_time ASC;
```

INNER JOIN

The query result is given below:

Results | Messages

	first_name	last_name	start_time	end_time	outcome_text
1	Thomas (Neo)	Anderson	2020-01-11 09:00:15.000	2020-01-11 09:12:22.000	finished - successfully
2	Agent	Smith	2020-01-11 09:02:20.000	2020-01-11 09:18:05.000	finished - unsuccessfully
3	Thomas (Neo)	Anderson	2020-01-11 09:14:50.000	2020-01-11 09:20:01.000	finished - successfully
4	Thomas (Neo)	Anderson	2020-01-11 09:24:15.000	2020-01-11 09:25:05.000	finished - unsuccessfully
5	Thomas (Neo)	Anderson	2020-01-11 09:26:23.000	2020-01-11 09:33:45.000	finished - successfully
6	Thomas (Neo)	Anderson	2020-01-11 09:40:31.000	2020-01-11 09:42:32.000	finished - successfully
7	Agent	Smith	2020-01-11 09:41:17.000	2020-01-11 09:45:21.000	finished - successfully
8	Thomas (Neo)	Anderson	2020-01-11 09:42:32.000	2020-01-11 09:46:53.000	finished - unsuccessfully
9	Agent	Smith	2020-01-11 09:46:00.000	2020-01-11 09:48:02.000	finished - successfully
10	Agent	Smith	2020-01-11 09:50:12.000	2020-01-11 09:55:35.000	finished - successfully

INNER JOIN

Join Operations

Join operation denoted by ⋈.

JOIN operation also allows joining variously related tuples from different relations.

Types of JOIN:

Various forms of join operation are:

Inner Joins:

- Theta join
- EQUI join
- Natural join

INNER JOIN

Theta Join:

The general case of JOIN operation is called a Theta join. It is denoted by symbol **θ**

Example

$$A \bowtie_{\theta} B$$

Theta join can use any conditions in the selection criteria.

For example:

$$A \bowtie_{A.column\ 2\ >\ B.column\ 2} (B)$$

INNER JOIN

EQUI join:

When a theta join uses only equivalence condition, it becomes a equi join.

For example:

$$A \bowtie_{A.column\ 2 = B.column\ 2} (B)$$

NATURAL JOIN (⋈)

Natural join can only be performed if there is a common attribute (column) between the relations. The name and type of the attribute must be same.

Example

Consider the following two tables

C

Num	Square
2	4
3	9

D

Num	Cube
2	8
3	27

```
C ⋈ D
```

C ⋈ D

Num	Square	Cube
2	4	8
3	9	27

SQL Query Patterns and DML Practice

Lecture-ready slides for classroom delivery

FILTER GROUP JOIN DML

- Core query families students must recognize
- Worked syntax for single-table, joins, and subqueries
- Practice prompts for INSERT, UPDATE, and DELETE

FROM / JOIN WHERE GROUP BY HAVING

```sql
SELECT DeptID, COUNT(*) AS
TotalEmployees
FROM Employee
WHERE Salary >= 50000
GROUP BY DeptID
HAVING COUNT(*) >= 1
ORDER BY TotalEmployees DESC;
```

Tip for lecture: first teach syntax order, then explain execution flow.

Lecture roadmap

This deck moves from reading data to modifying data.

1

Single-table filters

- WHERE
- IN
- BETWEEN
- LIKE

2

Aggregates

- COUNT / SUM / AVG
- Alias
- GROUP BY
- HAVING + ORDER BY

3

Relationship queries

- INNER JOIN
- LEFT / RIGHT
- FULL OUTER idea
- UNION

4

Subqueries

- IN (subquery)
- EXISTS
- ALL

5

DML practice

- INSERT
- UPDATE
- DELETE
- Prediction questions

Teaching cue: whenever students see aggregates, ask “Am I filtering rows or filtering groups?”

Example data model used across the lecture

Three related tables give enough coverage for filtering, joins, subqueries, UNION, and DML.

Department

DeptID	DeptName	Location
1	IT	Lahore
2	HR	Karachi
3	Sales	Islamabad
4	Finance	Lahore

Department.DeptID = Employee.DeptID

Employee

EmpID	EmpName	DeptID	Salary	City
101	Ali	1	50000	Lahore
102	Sara	2	60000	Karachi
103	Ahmed	1	55000	Lahore
104	Hina	3	45000	Faisalabad
105	Usman	2	70000	Lahore
106	Areeba	3	48000	Karachi
107	Bilal	4	65000	Lahore

Project

ProjectID	ProjectName	EmpID	Budget
1	Website	101	200000
2	Payroll	102	150000
3	App	103	300000
4	Marketing	104	120000
5	Audit	107	250000

Employee.EmpID = Project.EmpID

Teaching point: Usman and Areeba have no matching project rows, so LEFT JOIN produces NULL for ProjectName.

Query clause order students must memorize

CORE IDEA

Write clauses in one order, but explain logical processing in another.

Syntax order (how we write the query)

SELECT FROM JOIN WHERE GROUP BY HAVING ORDER BY

Logical processing order (how the DBMS reasons)

FROM / JOIN WHERE GROUP BY HAVING SELECT ORDER BY

Fast classroom rule: WHERE filters rows, HAVING filters groups.

Single-table filtering patterns

Start with row selection before introducing groups or joins.

WHERE

```
1 SELECT EmpName, Salary
2 FROM Employee
3 WHERE Salary >= 60000;
```

Condition on one or more columns.

IN

```
1 SELECT EmpName, City
2 FROM Employee
3 WHERE City IN
('Lahore', 'Karachi');
```

Useful when the same column accepts several allowed values.

BETWEEN

```
1 SELECT EmpName, Salary
2 FROM Employee
3 WHERE Salary BETWEEN
50000 AND 65000;
```

Inclusive range: both boundary values are included.

LIKE

```
1 SELECT EmpName
2 FROM Employee
3 WHERE EmpName LIKE 'A
%';
```

Pattern match. A% means names that start with A.

Aggregate functions, alias, GROUP BY, HAVING, ORDER BY

This is the slide to pause on and solve line by line.

```sql
SELECT DeptID,
       COUNT(*) AS TotalEmployees,
       AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY DeptID
HAVING COUNT(*) >= 2
ORDER BY AvgSalary DESC;
```

- Alias names the output columns: TotalEmployees, AvgSalary
- GROUP BY forms one result row per department
- HAVING keeps only grouped results with at least 2 rows
- ORDER BY sorts the final output after aggregation

Expected result

DeptID	TotalEmployees	AvgSalary
2	2	65000
1	2	52500
3	2	46500

Teaching cue: ask students why DeptID 4 disappeared. Answer: HAVING COUNT(*) >= 2 removed the one-row group.

One combined query: WHERE + GROUP BY + HAVING + ORDER BY

PIPELINE

Use this when you want to show the full pipeline in one example.

```sql
SELECT DeptID, COUNT(*) AS TotalEmployees, AVG(Salary) AS AvgSalary
FROM Employee
WHERE Salary >= 50000
GROUP BY DeptID
HAVING COUNT(*) >= 1
ORDER BY AvgSalary DESC;
```

Step 1: WHERE

Keep only salaries 50000 or above: Ali, Sara, Ahmed, Usman, Bilal.

Step 2: GROUP BY

Dept 1 = 2 rows, Dept 2 = 2 rows, Dept 4 = 1 row.

Step 3: HAVING

Every group stays because each grouped count is at least 1.

Step 4: ORDER BY

Sort by average salary descending.

Filtered rows

EmpName	DeptID	Salary
Ali	1	50000
Sara	2	60000
Ahmed	1	55000
Usman	2	70000
Bilal	4	65000

Final result

DeptID	TotalEmployees	AvgSalary
2	2	65000
4	1	65000
1	2	52500

Note: ties in AvgSalary can appear unless you add a second ORDER BY key.

Join family: INNER, LEFT, RIGHT, and FULL OUTER idea

Use employees with and without projects so NULLs become visible.

INNER JOIN

```sql
SELECT E.EmpName, P.ProjectName
FROM Employee E
INNER JOIN Project P
ON E.EmpID = P.EmpID;
```

Only matched rows from both tables.

LEFT JOIN

```sql
SELECT E.EmpName, P.ProjectName
FROM Employee E
LEFT JOIN Project P
ON E.EmpID = P.EmpID;
```

All employee rows. Missing project becomes NULL.

RIGHT JOIN

```sql
SELECT E.EmpName, P.ProjectName
FROM Employee E
RIGHT JOIN Project P
ON E.EmpID = P.EmpID;
```

All project rows. Useful when the right table must be preserved.

FULL OUTER

```sql
-- Standard SQL
SELECT E.EmpName, P.ProjectName
FROM Employee E
FULL OUTER JOIN Project P
ON
```

All rows from both sides. In MySQL, simulate with LEFT JOIN UNION RIGHT JOIN.

Subquery patterns: IN, EXISTS, ALL

Teach what the subquery returns before reading the outer query.

IN

```
SELECT EmpName
FROM Employee
WHERE DeptID IN
(SELECT DeptID
 FROM Department
 WHERE Location =
'Lahore');
```

Ask: which DeptID values come back from the subquery? Then match those values.

EXISTS

```
SELECT EmpName
FROM Employee E
WHERE EXISTS
(SELECT *
 FROM Project P
 WHERE P.EmpID =
E.EmpID);
```

EXISTS checks whether at least one matching row is found.

ALL

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary > ALL
(SELECT Salary
 FROM Employee
 WHERE DeptID = 3);
```

Compare against every value returned. Here salary must exceed both Sales salaries.

UNION versus UNION ALL

Good for combining compatible result sets from different queries.

UNION

```sql
SELECT City AS Place
FROM Employee
UNION
SELECT Location AS Place
FROM Department;
```

UNION removes duplicates automatically.

UNION ALL

```sql
SELECT City AS Place
FROM Employee
UNION ALL
SELECT Location AS Place
FROM Department;
```

UNION ALL keeps duplicates and is usually faster.

Example result with UNION

Place
Lahore
Karachi
Faisalabad
Islamabad

Compatibility rule for both queries:

- Same number of output columns
- Compatible data types
- Final ORDER BY, if used, comes once at the end

DML overview: INSERT, UPDATE, DELETE

When rows change, students should always check the target rows before executing.

INSERT

```
1 INSERT INTO Employee
2 (EmpID, EmpName,
DeptID, Salary, City)
3 VALUES (108, 'Mina',
1, 52000, 'Multan');
```

- Adds new rows
- Column list is safer than relying on table order

UPDATE

```
1 UPDATE Employee
2 SET Salary = Salary +
5000
3 WHERE DeptID = 3;
```

- Modifies existing rows
- Never skip WHERE unless every row must change

DELETE

```
1 DELETE FROM Employee
2 WHERE City =
'Karachi';
```

- Removes rows
- Always preview with SELECT first

DML worked examples students can predict before execution

WORKED

A good lecture rhythm: show the row(s), run the statement, then verify the output.

INSERT example

```
1 INSERT INTO Employee
2 (EmpID, EmpName, DeptID,
Salary, City)
3 VALUES (108, 'Mina', 1, 52000,
'Multan');
```

Question to class: How many IT / DeptID 1 employees exist after this insert?

UPDATE example

```
1 UPDATE Employee
2 SET Salary = Salary + 5000
3 WHERE DeptID = 3;
```

Affected rows: Hina 45000→50000, Areeba 48000→53000.

DELETE example

```
1 DELETE FROM Employee
2 WHERE City = 'Karachi';
```

Question to class: How many employee rows remain after deleting Sara and Areeba?

Practice questions: predict the output

PRACTICE

Use these as in-class pauses or quick quizzes.

```
1    SELECT          FROM          WHERE          =  ;
```

```
1    SELECT       (*) AS       FROM          WHERE       IN (          ,          );
```

```
1    SELECT          FROM          WHERE          LIKE     ;
```

```
1  SELECT       ,     (*) AS
2  FROM
3  GROUP BY
4  HAVING     (*) >
5  ORDER BY            DESC
```

```
1    SELECT  .          ,  .
2    FROM            LEFT JOIN
3    ON   .       =   .      ;
```

Teacher move: ask students to state the filtering step first, not the final answer immediately.

Practice questions: DML + combined queries

PRACTICE

These prompts are designed for board work, pair work, or lab tasks.

DML tasks

- Insert employee (109, Zoya, DeptID 2, 58000, Lahore).
- Increase every Finance employee salary by 3000.
- Delete employees whose city is Faisalabad.
- Update Mina city from Multan to Karachi.
- Delete projects whose budget is below 150000.

Query-writing tasks

- List employees with salary between 50000 and 70000.
- Show departments having more than one employee.
- Find employees who have a project using EXISTS.
- Show employee names and project names using LEFT JOIN.
- Write one query using WHERE, GROUP BY, HAVING, ORDER BY.

One-slide recap for students

A compact revision slide at the end of the lecture.

Row filters

- WHERE → filter rows
- IN → list of allowed values
- BETWEEN → inclusive range
- LIKE → pattern matching

Grouped results

- Aggregate functions: COUNT, SUM, AVG, MIN, MAX
- GROUP BY creates one row per group
- HAVING filters grouped output
- ORDER BY sorts final result

Joins & sets

- INNER → matched rows only
- LEFT → all left rows + matches
- RIGHT → all right rows + matches
- FULL OUTER → both sides (or LEFT UNION RIGHT in MySQL)
- UNION removes duplicates

DML safety

- INSERT adds rows
- UPDATE changes rows
- DELETE removes rows
- Preview target rows with SELECT before running
- Be careful with missing WHERE

```
SELECT ...
FROM ...
JOIN ...
WHERE ...
GROUP BY ...
HAVING ...
ORDER BY ...;
```